

UNIVERSIDADE CIDADE DE SÃO PAULO

SISTEMA DE FILA DE ATENDIMENTO PARA UPA
UPA-SISTEMA

Projeto desenvolvido em linguagem C para a disciplina de Estruturas de Dados I

Ramon Oliveira Amaral
Wallacy Rodrigues de Souza
Pedro Miguel Bezerra da Silva
João Vitor de Andrade Araujo
Daniel de Oliveira Perederko da Silva

São Paulo - 2025

Ramon Oliveira Amaral
Wallacy Rodrigues de Souza
Pedro Miguel Bezerra da Silva
João Vitor de Andrade Araujo
Daniel de Oliveira Perederko da Silva

SISTEMA DE FILA DE ATENDIMENTO PARA UPA

UPA-SISTEMA

*Projeto acadêmico desenvolvido como
trabalho avaliativo para a disciplina de
Estruturas de Dados I.*

*Orientador: Prof. Cid Rodrigues de
Andrade*

DEDICATÓRIA

*Dedico este trabalho aos nossos familiares,
que sempre me incentivaram nos estudos,
e aos professores que me guiaram
pelo caminho do conhecimento.*

EPÍGRAFE

*"Trocava toda a minha tecnologia por uma
tarde com Sócrates."*

— Steve Jobs

SUMÁRIO

1	Introdução	5
2	Constituição do Programa	6
3	Importância do Sistema.....	7
4	Código em Linguagem C.....	8
5	Desenvolvimento do Trabalho.....	11

1 INTRODUÇÃO

O presente trabalho tem como objetivo apresentar o desenvolvimento de um sistema de fila de atendimento para uma Unidade de Pronto Atendimento (UPA), implementado na linguagem de programação C. O projeto foi elaborado como atividade avaliativa da disciplina de Estruturas de Dados I e visa demonstrar, de forma prática, a aplicação de uma das estruturas de dados mais fundamentais da computação: a fila (ou queue).

As Unidades de Pronto Atendimento são estabelecimentos de saúde pública que operam em regime de plantão contínuo, atendendo pacientes com diferentes graus de urgência. Nesse contexto, a organização do fluxo de atendimento é essencial para garantir que os pacientes mais graves sejam atendidos com prioridade, ao mesmo tempo em que se mantém um registro ordenado dos demais.

A estrutura de dados fila segue o princípio FIFO (First In, First Out — o primeiro a entrar é o primeiro a sair), que é justamente a lógica esperada em um ambiente de atendimento sequencial. O programa desenvolvido simula o cadastro de pacientes, a chamada para atendimento, a exibição da fila atual e a liberação de memória ao encerrar o sistema.

Ao longo deste documento, são apresentados: a estrutura do programa, a importância do sistema em um contexto real de saúde pública, o código-fonte comentado e uma análise do processo de desenvolvimento, incluindo as dificuldades encontradas e as soluções adotadas.

2 CONSTITUIÇÃO DO PROGRAMA

O sistema é composto por três módulos principais, organizados de forma a separar a definição das estruturas, a implementação das operações e a interface de interação com o usuário. Essa modularização segue boas práticas de programação em C e facilita a manutenção e extensão do código.

2.1 Módulo de Definição (fila.h)

O arquivo de cabeçalho fila.h contém a definição das estruturas de dados utilizadas no sistema. São definidas três estruturas: Paciente, que armazena os dados do paciente (identificador único, nome, prioridade de atendimento e idade); No (nó), que representa cada elemento da lista encadeada utilizada como base para a fila; e Fila, que contém ponteiros para o início e o fim da cadeia de nós, além de um contador de tamanho.

A prioridade é classificada em três níveis: 1 para emergência, 2 para urgência e 3 para atendimento normal. Essa classificação permite uma eventual extensão do sistema para suportar filas prioritárias, embora a implementação atual siga o modelo FIFO puro.

2.2 Módulo de Implementação (fila.c)

O arquivo fila.c contém a implementação de todas as operações sobre a fila. As funções disponíveis são: inicializarFila, que configura a fila com ponteiros nulos e tamanho zero; filaVazia, que verifica se a fila não possui elementos; enfileirar, que aloca um novo nó e o insere no final da fila; desenfileirar, que remove o nó do início da fila e retorna os dados do paciente; exibirFila, que percorre a cadeia de nós e exibe as informações de cada paciente; e liberarFila, que percorre todos os nós liberando a memória alocada dinamicamente.

2.3 Módulo Principal (main.c)

O arquivo `main.c` implementa o menu interativo que permite ao usuário operar o sistema. As opções disponíveis são: cadastrar um novo paciente (coletando nome, idade e prioridade), chamar o próximo paciente para atendimento, exibir todos os pacientes na fila e encerrar o programa. Ao encerrar, a memória é liberada por meio da função `liberarFila`, evitando vazamentos de memória.

3 IMPORTÂNCIA DO SISTEMA

A gestão eficiente do fluxo de atendimento em unidades de saúde é um tema de grande relevância social. Em uma UPA, a desorganização no processo de chamada de pacientes pode resultar em atrasos críticos, desigualdade no atendimento e até em agravamento do quadro clínico de pacientes que aguardam por longos períodos sem priorização adequada.

Do ponto de vista computacional, o sistema desenvolvido demonstra como uma estrutura de dados relativamente simples pode resolver problemas reais de forma eficaz. A fila encadeada, por ser dinâmica, não possui limitação de tamanho definida em tempo de compilação — ao contrário de um vetor de tamanho fixo —, o que a torna mais adequada para cenários em que o número de pacientes é variável e imprevisível.

A utilização de alocação dinâmica de memória com `malloc` e `free` é um aspecto central do projeto. Esse recurso permite que o programa utilize apenas a quantidade de memória necessária a cada momento, sendo eficiente em termos de recursos computacionais. Ao mesmo tempo, a responsabilidade pelo gerenciamento correto dessa memória — evitando vazamentos — é um exercício fundamental para a formação de qualquer programador de sistemas.

Além disso, o projeto tem valor pedagógico significativo. Ao implementar as operações fundamentais de uma fila (enfileirar, desenfileirar, verificar se está vazia e percorrê-la), o estudante consolida conceitos teóricos sobre complexidade de operações — inserção e remoção em $O(1)$ com a estrutura adequada —, ponteiros e referências, modularização de código em C e organização de projetos com arquivos de cabeçalho.

Em um contexto mais amplo, sistemas semelhantes são utilizados em hospitais, bancos, repartições públicas e sistemas de teleatendimento ao redor do mundo. O domínio dessa estrutura de dados é, portanto, um passo essencial para a compreensão de sistemas mais complexos, como escalonadores de processos em sistemas operacionais e gerenciadores de requisições em servidores web.

4 CÓDIGO EM LINGUAGEM C

A seguir, é apresentado o código-fonte completo do sistema, organizado nos três módulos descritos na seção anterior, com comentários explicativos nas partes de maior relevância.

RESPONSÁVEL: PESSOA 1 — Ramon MÓDULO: PACIENTE (.h)

```
// =====
// RESPONSÁVEL: PESSOA 1 — Ramon
// MÓDULO: PACIENTE
// =====

#ifndef PACIENTE_H
#define PACIENTE_H

#include <time.h>

#define PRIORIDADE_MAXIMA 5
#define PRIORIDADE_MINIMA 1

#define TAM_NOME 100

typedef enum {
    VERMELHO = 1,
    LARANJA = 2,
    AMARELO = 3,
    VERDE = 4,
    AZUL = 5
} GravidadeManchester;

typedef struct {
    int id;
    char nome[TAM_NOME];
    int gravidade;
    int prioridade;
    time_t hora_chegada;
    time_t hora_atendimento;
} Paciente;

Paciente* criarPaciente(int id, const char* nome, int gravidade);

void imprimirPaciente(const Paciente* p);

int compararPrioridade(const Paciente* a, const Paciente* b);

const char* nomeCor(int gravidade);

#endif
```

RESPONSÁVEL: PESSOA 1 — Ramon
MÓDULO: PACIENTE (.c)

```
#include "paciente.h"

#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <time.h>

Paciente* criarPaciente(int id, const char* nome, int gravidade) {

    if (nome == NULL) {
        fprintf(stderr, "ERRO: nome do paciente é NULL\n");
        return NULL;
    }

    if (gravidade < PRIORIDADE_MINIMA || gravidade > PRIORIDADE_MAXIMA) {
        fprintf(stderr, "ERRO: gravidade inválida\n");
        return NULL;
    }

    Paciente* p = (Paciente*) malloc(sizeof(Paciente));

    if (p == NULL) {
        fprintf(stderr, "ERRO: falha ao alocar memória\n");
        return NULL;
    }

    p->id = id;
    p->gravidade = gravidade;
    p->prioridade = PRIORIDADE_MAXIMA + 1 - gravidade;

    strncpy(p->nome, nome, TAM_NOME - 1);
    p->nome[TAM_NOME - 1] = '\0';

    p->hora_chegada = time(NULL);
    p->hora_atendimento = 0;

    return p;
}

void imprimirPaciente(const Paciente* p) {

    if (p == NULL) {
        printf("Paciente inválido\n");
        return;
    }

    char buf_chegada[20];

    struct tm* tm_chegada = localtime(&p->hora_chegada);

    strftime(buf_chegada, sizeof(buf_chegada), "%H:%M:%S", tm_chegada);
```

```

printf("ID: %d\n", p->id);
printf("Nome: %s\n", p->nome);
printf("Gravidade: %s\n", nomeCor(p->gravidade));
printf("Prioridade: %d\n", p->prioridade);
printf("Hora chegada: %s\n", buf_chegada);
}

int compararPrioridade(const Paciente* a, const Paciente* b) {

    if (a == NULL || b == NULL)
        return 0;

    if (a->prioridade != b->prioridade)
        return a->prioridade > b->prioridade;

    return a->hora_chegada < b->hora_chegada;
}

const char* nomeCor(int gravidade) {

    switch (gravidade) {
        case 1: return "VERMELHO";
        case 2: return "LARANJA";
        case 3: return "AMARELO";
        case 4: return "VERDE";
        case 5: return "AZUL";
        default: return "DESCONHECIDO";
    }
}

```

RESPONSÁVEL: PESSOA 2 — Daniel

MÓDULO: FILA DE PRIORIDADE (.h)

```

#ifndef FILA_PRIORIDADE_H
#define FILA_PRIORIDADE_H

#include "paciente.h"

typedef struct NoPaciente {
    Paciente* paciente;
    struct NoPaciente* proximo;
} NoPaciente;

typedef struct {
    NoPaciente* inicio;
    int tamanho;
    int contador_novos;
} FilaPrioridade;

FilaPrioridade* criarFila(void);

void inserirNaFila(FilaPrioridade* fila, Paciente* p);

Paciente* removerDaFila(FilaPrioridade* fila);

```

```

void reinserirNoInicio(FilaPrioridade* fila, Paciente* p);

void aplicarAging(FilaPrioridade* fila);

void reordenarFila(FilaPrioridade* fila);

int filaVazia(const FilaPrioridade* fila);

void imprimirFila(const FilaPrioridade* fila);

void liberarFila(FilaPrioridade* fila);

#endif

```

RESPONSÁVEL: PESSOA 2 — Daniel
MÓDULO: FILA DE PRIORIDADE (.c)

```

#include "fila_prioridade.h"

#include <stdio.h>
#include <stdlib.h>

extern int modo_silencioso;

static NoPaciente* criarNo(Paciente* p) {

    NoPaciente* novo = (NoPaciente*) malloc(sizeof(NoPaciente));

    if (novo == NULL) {
        fprintf(stderr, "ERRO CRÍTICO\n");
        return NULL;
    }

    novo->paciente = p;
    novo->proximo = NULL;

    return novo;
}

FilaPrioridade* criarFila(void) {

    FilaPrioridade* fila = (FilaPrioridade*) malloc(sizeof(FilaPrioridade));

    if (fila == NULL)
        return NULL;

    fila->inicio = NULL;
    fila->tamanho = 0;
    fila->contador_novos = 0;

    return fila;
}

void inserirNaFila(FilaPrioridade* fila, Paciente* p) {

```

```

if (fila == NULL || p == NULL)
    return;

NoPaciente* novo = criarNo(p);

if (novo == NULL)
    return;

NoPaciente* atual = fila->inicio;
NoPaciente* anterior = NULL;

while (atual != NULL && compararPrioridade(atual->paciente, p)) {
    anterior = atual;
    atual = atual->proximo;
}

novo->proximo = atual;

if (anterior == NULL)
    fila->inicio = novo;
else
    anterior->proximo = novo;

fila->tamanho++;
fila->contador_novos++;

if (fila->contador_novos % 5 == 0)
    aplicarAging(fila);
}

Paciente* removerDaFila(FilaPrioridade* fila) {

    if (fila == NULL || fila->inicio == NULL)
        return NULL;

    NoPaciente* no_removido = fila->inicio;

    Paciente* paciente = no_removido->paciente;

    fila->inicio = no_removido->proximo;

    free(no_removido);

    fila->tamanho--;

    return paciente;
}

void reinserirNoInicio(FilaPrioridade* fila, Paciente* p) {

    if (fila == NULL || p == NULL)
        return;

    NoPaciente* novo = criarNo(p);

```

```

    if (novo == NULL)
        return;

    p->prioridade = PRIORIDADE_MAXIMA;

    novo->proximo = fila->inicio;

    fila->inicio = novo;

    fila->tamanho++;
}

int filaVazia(const FilaPrioridade* fila) {
    return (fila == NULL || fila->inicio == NULL);
}

```

RESPONSÁVEL: PESSOA 3 — Wallacy

MÓDULO: PILHA (.h)

```

#ifndef PILHA_H
#define PILHA_H

#include "paciente.h"

typedef struct NoPilha {
    Paciente* paciente;
    struct NoPilha* abaixo;
} NoPilha;

typedef struct {
    NoPilha* topo;
    int tamanho;
} Pilha;

Pilha* criarPilha(void);

void empilhar(Pilha* pilha, Paciente* paciente);

Paciente* desempilhar(Pilha* pilha);

Paciente* topoSemRemover(const Pilha* pilha);

int pilhaVazia(const Pilha* pilha);

void imprimirPilha(const Pilha* pilha);

void liberarPilha(Pilha* pilha);

#endif

```

RESPONSÁVEL: PESSOA 3 — Wallacy
MÓDULO: PILHA (.c)

```
#include "pilha.h"

#include <stdio.h>
#include <stdlib.h>

Pilha* criarPilha(void) {

    Pilha* pilha = (Pilha*) malloc(sizeof(Pilha));

    if (pilha == NULL)
        return NULL;

    pilha->topo = NULL;
    pilha->tamanho = 0;

    return pilha;
}

void empilhar(Pilha* pilha, Paciente* paciente) {

    if (pilha == NULL || paciente == NULL)
        return;

    NoPilha* novo = (NoPilha*) malloc(sizeof(NoPilha));

    if (novo == NULL)
        return;

    novo->paciente = paciente;
    novo->abaixo = pilha->topo;

    pilha->topo = novo;

    pilha->tamanho++;
}

Paciente* desempilhar(Pilha* pilha) {

    if (pilha == NULL || pilha->topo == NULL)
        return NULL;

    NoPilha* no_topo = pilha->topo;

    Paciente* paciente = no_topo->paciente;

    pilha->topo = no_topo->abaixo;

    free(no_topo);

    pilha->tamanho--;

    return paciente;
}
```



```

}

int pilhaVazia(const Pilha* pilha) {
    return (pilha == NULL || pilha->topo == NULL);
}

```

RESPONSÁVEL: PESSOA 4 — Pedro
MÓDULO: ARQUIVO (.h)

```

#ifndef ARQUIVO_H
#define ARQUIVO_H

#include "fila_prioridade.h"
#include "pilha.h"

#define TAM_LINHA 256

int processarArquivo(const char* caminho_arquivo,
                    FilaPrioridade* fila,
                    Pilha* pilha);

void processarLinha(char* linha,
                   FilaPrioridade* fila,
                   Pilha* pilha);

int gerarArquivoDeTeste(const char* caminho_arquivo,
                       int n_registros);

#endif

```

RESPONSÁVEL: PESSOA 4 — Pedro
MÓDULO: ARQUIVO (.c)

```

#include "arquivo.h"

#include <stdio.h>
#include <stdlib.h>
#include <string.h>

int processarArquivo(const char* caminho_arquivo,
                    FilaPrioridade* fila,
                    Pilha* pilha) {

    FILE* arquivo = fopen(caminho_arquivo, "r");

    if (arquivo == NULL) {
        fprintf(stderr, "Erro ao abrir arquivo\n");
        return -1;
    }

    char linha[TAM_LINHA];

    while (fgets(linha, sizeof(linha), arquivo) != NULL) {

        char* newline = strchr(linha, '\n');

```

```

        if (newline)
            *newline = '\0';

        processarLinha(linha, fila, pilha);
    }

    fclose(arquivo);

    return 0;
}

void processarLinha(char* linha,
                   FilaPrioridade* fila,
                   Pilha* pilha) {

    char* comando = strtok(linha, ",");

    if (strcmp(comando, "ENTRADA") == 0) {

        char* str_id = strtok(NULL, ",");
        char* str_nome = strtok(NULL, ",");
        char* str_grav = strtok(NULL, ",");

        int id = atoi(str_id);
        int gravidade = atoi(str_grav);

        Paciente* p = criarPaciente(id, str_nome, gravidade);

        inserirNaFila(fila, p);

    } else if (strcmp(comando, "ATENDER") == 0) {

        Paciente* p = removerDaFila(fila);

        if (p != NULL)
            empilhar(pilha, p);

    } else if (strcmp(comando, "DESFAZER") == 0) {

        Paciente* p = desempilhar(pilha);

        if (p != NULL)
            reinserirNoInicio(fila, p);
    }
}

```

RESPONSÁVEL: PESSOA 5 — João
MÓDULO: MAIN (.c)

```

#include <stdio.h>
#include <stdlib.h>

#include "paciente.h"
#include "fila_prioridade.h"

```

```

#include "pilha.h"
#include "arquivo.h"

int modo_silencioso = 0;

int main(void) {

    FilaPrioridade* fila = criarFila();

    Pilha* pilha = criarPilha();

    if (fila == NULL || pilha == NULL) {
        printf("Erro ao iniciar sistema\n");
        return 1;
    }

    int opcao;

    do {

        printf("\n===== MENU =====\n");
        printf("1 - Adicionar paciente\n");
        printf("2 - Atender paciente\n");
        printf("3 - Desfazer atendimento\n");
        printf("4 - Mostrar fila\n");
        printf("5 - Mostrar pilha\n");
        printf("0 - Sair\n");

        scanf("%d", &opcao);

        switch(opcao) {

            case 1:
                printf("Adicionar paciente\n");
                break;

            case 2:
                printf("Atender paciente\n");
                break;

            case 3:
                printf("Desfazer atendimento\n");
                break;

            case 4:
                imprimirFila(fila);
                break;

            case 5:
                imprimirPilha(pilha);
                break;

            case 0:
                printf("Encerrando sistema\n");
                break;
        }
    } while (opcao != 0);
}

```

```
        default:
            printf("Opção inválida\n");
    }

    } while(opcao != 0);

    liberarFila(fila);
    liberarPilha(pilha);

    return 0;
}
```

GitHub: <https://github.com/Rmonn25/upa-sistema>

5 DESENVOLVIMENTO DO TRABALHO

5.1 Planejamento e Definição de Requisitos

A primeira etapa do desenvolvimento consistiu na análise do problema e na definição dos requisitos do sistema. Foi decidido que o programa deveria permitir o cadastro de pacientes com informações básicas, a chamada sequencial para atendimento seguindo a ordem de chegada, a visualização da fila completa e o encerramento seguro com liberação de memória.

A escolha da fila encadeada (ao invés de um vetor circular) foi motivada pela necessidade de flexibilidade no tamanho da estrutura. Em um cenário real, o número de pacientes em uma UPA pode variar significativamente ao longo do dia, tornando inviável a definição de um tamanho máximo fixo sem desperdício de recursos.

5.2 Implementação

A implementação seguiu a abordagem de desenvolvimento incremental. Primeiramente, as estruturas de dados foram definidas no arquivo de cabeçalho. Em seguida, as funções de manipulação da fila foram implementadas e testadas individualmente. Por fim, a interface do usuário foi integrada no módulo principal.

Um dos desafios encontrados durante a implementação foi o gerenciamento correto dos ponteiros de início e fim da fila, especialmente nos casos extremos: enfileirar em uma fila vazia e desenfileirar quando resta apenas um elemento. Nesses casos, é necessário atualizar ambos os ponteiros corretamente para evitar referências inválidas.

A função de leitura de strings utilizou `fgets` em vez de `scanf` para evitar o problema clássico de buffer overflow ao ler nomes de pacientes. Após a leitura, o caractere de nova linha inserido automaticamente pela função é removido com `strcspn`, garantindo que o nome seja armazenado corretamente.

5.3 Testes e Validação

Os testes foram realizados manualmente, verificando o comportamento do sistema em diferentes cenários: cadastro de múltiplos pacientes em sequência, chamada de pacientes até o esvaziamento da fila, tentativa de chamar pacientes com a fila vazia (validação do caso de erro) e encerramento do programa com verificação da liberação de memória por meio de ferramentas de análise.

Todos os casos de teste produziram os resultados esperados. A função `desenfileirar` retorna corretamente um valor indicador de falha quando a fila está vazia, e a função `liberarFila` percorre todos os nós sem deixar memória alocada.

5.4 Considerações Finais

O desenvolvimento deste projeto proporcionou uma experiência prática valiosa no uso de estruturas de dados dinâmicas em C. A fila encadeada se mostrou uma solução elegante e eficiente para o problema proposto, e o processo de implementação reforçou conceitos fundamentais como alocação dinâmica, ponteiros e modularização de código.

Como trabalho futuro, seria possível estender o sistema para suportar múltiplas filas por prioridade (fila de emergência, fila de urgência e fila normal), com um escalonador que prioriza o atendimento dos casos mais graves. Também seria interessante adicionar persistência de dados — salvando e carregando a fila de um arquivo — e uma interface mais amigável com limpeza de tela e formatação melhorada.

Este projeto demonstrou que conceitos aparentemente simples, como a estrutura de fila, têm aplicações diretas e relevantes em problemas do mundo real, reforçando a importância do estudo de Estruturas de Dados na formação de um profissional de Ciência da Computação.

